

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 8



Oleh :

M.YAZEM AGVA ROIZ

NIM 2511533003

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2026

Penulis

DAFTAR ISI

BAB 1.....	4
1. Latar Belakang.....	4
2. Tujuan.....	4
3. Manfaat.....	5
BAB 2.....	5
A. Teori.....	5
B. Code.....	5
BAB 3.....	19
a. Kesimpulan.....	19
b. Saran.....	19

BAB 1

PENDAHULUAN

1. Latar Belakang

Algoritma merupakan cara untuk menyelesaikan suatu masalah secara sistematis, bertahap dan jelas, pemrograman sendiri menggunakan algoritma dalam pembuatan program dan otomatisasi masalah, karena itu algoritma menjadi bagian yang penting dalam pemrograman itu sendiri.

Pemahaman Algoritma sendiri meningkatkan efisiensi dan optimalisasi suatu program, pemilihan algoritma memerlukan pemahaman yang baik dalam kasus yang dihadapi, oleh karena itu pembelajaran program diperlukan terutama pada jurusan informatika yang berkecimpung pada dunia teknologi.

Salah satu kasus penggunaan algoritma adalah algoritma sorting yang bekerja dalam mengurutkan angka yang tidak terurut menjadi berurut. Dalam praktikum kali ini akan mempraktekan algoritma quick sort, merge sort, shell sort serta visualisasi dari bubble sort dan merge sort dalam bentuk GUI.

2. Tujuan

Tujuan dari praktikum adalah memahami dan mengetahui bagaimana alur dari algoritma yang berfungsi dalam mengurutkan bilangan yang acak secara teori, alur dan prakteknya melalui code secara langsung.

3. Manfaat

Manfaat dalam membuat praktikum ini adalah mendapatkan pemahaman secara teori dan praktek dalam pembuatan bermacam algoritma sorting.

BAB 2

PEMBAHASAN

A. Teori

Algoritma pemrograman adalah seni dan ilmu dalam menyusun langkah-langkah terstruktur untuk membentuk suatu program. Dalam kasus angka yang tidak terurut terdapat banyak algoritma yang memiliki tingkat efisiensi berbeda dan kasus kasus terburuk dan terbaik yang berbeda pula, dalam praktikum ini akan membahas algoritma sorting antara lain:

1. Merge Sort
2. Shell Sort
3. Quick Sort

B. Code

a. ShellSort_2511533003.java

```
package pekan8_2511533003;
public class ShellSort_2511533003 {
    public static void shellSort(int[] A_3003) {
        int n_3003 = A_3003.length;
        int gap_3003 = n_3003 / 2;
        while (gap_3003 > 0) {
            for (int i_3003 = gap_3003; i_3003 < n_3003; i_3003++) {
                int temp_3003 = A_3003[i_3003];
                int j_3003 = i_3003;
                while (
                    j_3003 >= gap_3003 && A_3003[j_3003 - gap_3003] > temp_3003
                ) {
                    A_3003[j_3003] = A_3003[j_3003 - gap_3003];
                    j_3003 = j_3003 - gap_3003;
                }
                A_3003[j_3003] = temp_3003;
            }
            gap_3003 = gap_3003 / 2;
        }
    }
}
```

```

    }

    public static void printArray_3003(int[] arr_3003) {
        for (int i_3003 : arr_3003) {
            System.out.print(i_3003 + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] data_3003 = { 3, 10, 4, 6, 8, 9, 7, 2, 1, 5 };

        System.out.print("Sebelum: ");
        printArray_3003(data_3003);

        shellSort(data_3003);

        System.out.print("Sesudah (ShellSort): ");
        printArray_3003(data_3003);
    }
}

```

Code diatas merupakan code program dalam mempraktikan alur shell sort dalam mengurutkan angka acak, shell sort sendiri mirip dengan insertion sort namun perbedaan berada pada penggunaan gap sebagai penentu angka yang akan dibandingkan. Penggunaan Gap inilah membuat shell sort menjadi efisien karena menjauhi kasus terburuknya hingga pada gap 1 yang dimana angka sudah mulai terurut membuat tahapan ini menjadi lebih cepat.

b. QuickSort_2511533003.java

```

package pekan8_2511533003;
public class QuickSort_2511533003 {
    static void swap_3003(int[] arr_3003, int i_3003, int j_3003) {
        int temp_3003 = arr_3003[i_3003];
        arr_3003[i_3003] = arr_3003[j_3003];
        arr_3003[j_3003] = temp_3003;
    }

    static void medianOfThree_3003(
        int[] arr_3003,
        int low_3003,
        int high_3003
    ) {

```

```

    ) {
        int mid_3003 = low_3003 + (high_3003 - low_3003) / 2;

        if (arr_3003[low_3003] > arr_3003[mid_3003]) {
            swap_3003(arr_3003, low_3003, mid_3003);
        }

        if (arr_3003[low_3003] > arr_3003[high_3003]) {
            swap_3003(arr_3003, low_3003, high_3003);
        }

        if (arr_3003[mid_3003] > arr_3003[high_3003]) {
            swap_3003(arr_3003, mid_3003, high_3003);
        }

        swap_3003(arr_3003, mid_3003, high_3003);
    }

    static int partition_3003(int[] arr_3003, int low_3003, int high_3003) {
        medianOfThree_3003(arr_3003, low_3003, high_3003);
        int pivot_3003 = arr_3003[high_3003];
        int i_3003 = (low_3003 - 1);

        for (int j_3003 = low_3003; j_3003 <= high_3003 - 1; j_3003++) {
            if (arr_3003[j_3003] < pivot_3003) {
                i_3003++;
                swap_3003(arr_3003, i_3003, j_3003);
            }
        }
        swap_3003(arr_3003, i_3003 + 1, high_3003);
        return i_3003 + 1;
    }

    public static void printArr_3003(int[] arr_3003) {
        for (int i_3003 : arr_3003) {
            System.out.print(i_3003 + " ");
        }
        System.out.println();
    }

    public static void quickSort_3003(
        int[] arr_3003,
        int low_3003,
        int high_3003
    ) {
        if (low_3003 < high_3003) {
            int pi_3003 = partition_3003(arr_3003, low_3003, high_3003);
            quickSort_3003(arr_3003, low_3003, pi_3003 - 1);
            quickSort_3003(arr_3003, pi_3003 + 1, high_3003);
        }
    }

```

```

    }
}

public static void main(String[] args) {
    int[] arr_3003 = { 3, 10, 4, 6, 8, 9, 7, 2, 1, 5 };
    int N_3003 = arr_3003.length;

    System.out.print("Data Sebelum Diurutkan: ");
    printArr_3003(arr_3003);

    quickSort_3003(arr_3003, 0, N_3003 - 1);

    System.out.print("Data Terurut quicksort: ");
    printArr_3003(arr_3003);
}
}

```

Quick sort adalah algoritma yang efisien dalam pengurutan, berjalan berdasarkan penentuan pivot dan pembagian berdasarkan sifat rekursif membuat algoritma ini secara average memiliki kompleksitas $n \log(n)$ yang membuat algoritma ini efisien.

Alur diawali dengan penentuan pivot, pada code program, pivot sendiri dipilih melalui median of three lalu media dari 3 angka dipilih (awal, tengah, akhir) akan di swap di akhir, pivot akan menjadi penentu perbandingan untuk setiap angka dan hingga akhir, pivot akan di swap dan menjadi penengah dengan pola sisi kiri akan lebih kecil dari pivot, sisi kanan akan lebih besar dari pivot, dan untuk sisi kanan dan kiri akan dilakukan hal yang sama.

c. MergeSort_2511533003.java

```

package pekan8_2511533003;
public class MergeSort_2511533003 {
    void merge_3003(int[] arr_3003, int l_3003, int m_3003, int
r_3003) {
        int n1_3003 = m_3003 - l_3003 + 1;
        int n2_3003 = r_3003 - m_3003;
    }
}

```



```

int L_3003[] = new int[n1_3003];
int R_3003[] = new int[n2_3003];

        for (int i_3003 = 0; i_3003 < n1_3003; ++i_3003)
L_3003[i_3003] =
        arr_3003[l_3003 + i_3003];

        for (int j_3003 = 0; j_3003 < n2_3003; ++j_3003)
R_3003[j_3003] =
        arr_3003[m_3003 + 1 + j_3003];

int i_3003 = 0,
    j_3003 = 0;

int k_3003 = l_3003;

while (i_3003 < n1_3003 && j_3003 < n2_3003) {
    if (L_3003[i_3003] <= R_3003[j_3003]) {
        arr_3003[k_3003] = L_3003[i_3003];
        i_3003++;
    } else {
        arr_3003[k_3003] = R_3003[j_3003];
        j_3003++;
    }
    k_3003++;
}

while (i_3003 < n1_3003) {
    arr_3003[k_3003] = L_3003[i_3003];
    i_3003++;
    k_3003++;
}

while (j_3003 < n2_3003) {
    arr_3003[k_3003] = R_3003[j_3003];
    j_3003++;
    k_3003++;
}

}

void sort_3003(int[] arr_3003, int l_3003, int r_3003) {

```

```

        if (l_3003 < r_3003) {
            int m_3003 = (l_3003 + r_3003) / 2;
            sort_3003(arr_3003, l_3003, m_3003);
            sort_3003(arr_3003, m_3003 + 1, r_3003);
            merge_3003(arr_3003, l_3003, m_3003, r_3003);
        }
    }

    public static void printArray_3003(int[] arr_3003) {
        int n_3003 = arr_3003.length;
        for (int i_3003 = 0; i_3003 < n_3003; i_3003++) {
            System.out.print(arr_3003[i_3003] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr_3003 = { 12, 11, 13, 5, 6, 7 };

        System.out.print("Sebelum Diurutkan: ");
        printArray_3003(arr_3003);

        MergeSort_2511533003 ob_3003 = new
MergeSort_2511533003();
        ob_3003.sort_3003(arr_3003, 0, arr_3003.length - 1);

        System.out.println("\nSesudah Terurut Menggunakan Merge
Sort: ");
        printArray_3003(arr_3003);
    }
}

```

Merge sort merupakan code sorting efisien dan stabil dalam mengurutkan, dengan alur membagi semua list angka menjadi partisi hingga pada satu titik setiap angka menjadi saling individual dan digabung dengan angka tetangganya, seperti itu terus secara rekursif hingga angka saling berurut.

d. BubbleSortGUI_2511533003

Bubble Sort GUI memiliki template yang sama dengan GUI pada pekan sebelumnya, oleh karena itu fungsi yang berbeda adalah pada fungsi yang bernama `setArrayFromInput_3003` yang berisi:

```
private void setArrayFromInput_3003() {
    String text_3003 = inputField_3003.getText().trim();
    if (text_3003.isEmpty()) return;
    String[] parts_3003 = text_3003.split(",");
    array_3003 = new int[parts_3003.length];

    try {
        for (int k_3003 = 0; k_3003 < parts_3003.length; k_3003++) {
            array_3003[k_3003] = Integer.parseInt(
                parts_3003[k_3003].trim()
            );
        }
    } catch (NumberFormatException e_3003) {
        JOptionPane.showMessageDialog(
            this,
            "Masukkan Hanya Angka " + "Yang dipisahkan koma!",
            "ERROR",
            JOptionPane.ERROR_MESSAGE
        );
        return;
    }
}

i_3003 = 0;
j_3003 = 0;
stepCount_3003 = 1;
sorting_3003 = true;
stepButton_3003.setEnabled(true);
stepArea_3003.setText("");
panelArray_3003.removeAll();
labelArray_3003 = new JLabel[array_3003.length];

for (int k_3003 = 0; k_3003 < array_3003.length; k_3003++) {
    labelArray_3003[k_3003] = new JLabel(
        String.valueOf(array_3003[k_3003])
    );
    labelArray_3003[k_3003].setFont(new Font("Arial", Font.BOLD, 24));
    labelArray_3003[k_3003].setOpaque(true);
    labelArray_3003[k_3003].setBackground(Color.WHITE);
    labelArray_3003[k_3003].setBorder(
        BorderFactory.createLineBorder(Color.BLACK)
    );
    labelArray_3003[k_3003].setPreferredSize(new Dimension(50, 50));
    labelArray_3003[k_3003].setHorizontalAlignment(
        SwingConstants.CENTER
    );
}
```

```

    );

    panelArray_3003.add(labelArray_3003[k_3003]);
}

panelArray_3003.revalidate();
panelArray_3003.repaint();
}

```

Code diatas merupakan code untuk input nilai array. Input akan berupa teks yang angka yang dipisahkan dengan koma, lalu setelah itu pada fungsi performStep_3003.

```

private void performStep_3003() {
    if (!sorting_3003 || i_3003 >= array_3003.length - 1) {
        sorting_3003 = false;
        stepButton_3003.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting Selesai!");
        return;
    }

    resetHighlights_3003();

    StringBuilder stepLog_3003 = new StringBuilder();
    labelArray_3003[j_3003].setBackground(Color.CYAN);
    labelArray_3003[j_3003 + 1].setBackground(Color.CYAN);

    if (array_3003[j_3003] > array_3003[j_3003 + 1]) {
        int temp_3003 = array_3003[j_3003];
        array_3003[j_3003] = array_3003[j_3003 + 1];
        array_3003[j_3003 + 1] = temp_3003;

        labelArray_3003[j_3003].setBackground(Color.RED);
        labelArray_3003[j_3003 + 1].setBackground(Color.RED);

        stepLog_3003
            .append("Langkah ")
            .append(stepCount_3003)
            .append(": ")
            .append("Menukar elemen ke-")
            .append(j_3003)
            .append(" (")
            .append(array_3003[j_3003 + 1])
            .append(")")
            .append(" dengan ke-")
            .append(j_3003 + 1)
            .append(" (")
            .append(array_3003[j_3003])

```

```

        .append("\n");
    } else {
        stepLog_3003
            .append("Langkah ")
            .append(stepCount_3003)
            .append(": ")
            .append("Tidak ada pertukaran antara ke-")
            .append(j_3003)
            .append(" dan ke-")
            .append(j_3003 + 1)
            .append("\n");
    }

    stepLog_3003
        .append("Hasil: ")
        .append(arrayToString_3003(array_3003))
        .append("\n\n");
    stepArea_3003.append(stepLog_3003.toString());

    updateLabels_3003();
    j_3003++;

    if (j_3003 >= array_3003.length - i_3003 - 1) {
        j_3003 = 0;
        i_3003++;
    }

    stepCount_3003++;

    if (i_3003 >= array_3003.length - 1) {
        sorting_3003 = false;
        stepButton_3003.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

```

performStep_3003 yang berguna untuk memperlihatkan tahapan tahapan dari sorting hingga angka sudah terurut semuanya.

e. MergeSortGUI_2511533003

Bubble Sort GUI memiliki template yang sama dengan GUI pada pekan sebelumnya, oleh karena itu fungsi yang berbeda adalah pada fungsi yang bernama `setArrayFromInput_3003` yang berisi:

```
private void setArrayFromInput_3003() {
    String text_3003 = inputField_3003.getText().trim();
    if (text_3003.isEmpty()) return;
    String[] parts_3003 = text_3003.split(",");
    array_3003 = new int[parts_3003.length];

    try {
        for (int k_3003 = 0; k_3003 < parts_3003.length; k_3003++) {
            array_3003[k_3003] = Integer.parseInt(
                parts_3003[k_3003].trim()
            );
        }
    } catch (NumberFormatException e_3003) {
        JOptionPane.showMessageDialog(
            this,
            "Masukkan Hanya Angka " + "Yang dipisahkan koma!",
            "ERROR",
            JOptionPane.ERROR_MESSAGE
        );
        return;
    }
}
```

```
labelArray_3003 = new JLabel[array_3003.length];
panelArray_3003.removeAll();
for (int k_3003 = 0; k_3003 < array_3003.length; k_3003++) {
    labelArray_3003[k_3003] = new JLabel(
```

```

        String.valueOf(array_3003[k_3003])
    );
    labelArray_3003[k_3003].setFont(new Font("Arial", Font.BOLD,
24));
    labelArray_3003[k_3003].setOpaque(true);
    labelArray_3003[k_3003].setBackground(Color.WHITE);
    labelArray_3003[k_3003].setBorder(
        BorderFactory.createLineBorder(Color.BLACK)
    );
    labelArray_3003[k_3003].setPreferredSize(new Dimension(50,
50));
    labelArray_3003[k_3003].setHorizontalAlignment(
        SwingConstants.CENTER
    );

    panelArray_3003.add(labelArray_3003[k_3003]);
}

mergeQueue_3003.clear();
generateMergeSteps_3003(0, array_3003.length - 1);
stepButton_3003.setEnabled(true);
stepArea_3003.setText("");
stepCount_3003 = 1;
isMerging_3003 = false;
panelArray_3003.revalidate();
panelArray_3003.repaint();
}

```

Code diatas merupakan code untuk input nilai array. Input akan berupa teks yang angka yang dipisahkan dengan koma, lalu setelah itu pada fungsi performStep_3003.

```
private void performStep_3003() {
    resetHighlights_3003();
    if (!isMerging_3003 && !mergeQueue_3003.isEmpty()) {
        int[] range_3003 = mergeQueue_3003.poll();
        left_3003 = range_3003[0];
        mid_3003 = range_3003[1];
        right_3003 = range_3003[2];
        temp_3003 = new int[right_3003 - left_3003 + 1];

        i_3003 = left_3003;
        j_3003 = mid_3003 + 1;
        k_3003 = 0;

        copying_3003 = false;
        isMerging_3003 = true;
        stepArea_3003.append(
            "Langkah " +
            stepCount_3003++ +
            ": Mulai merge dari " +
            left_3003 +
            " ke " +
            right_3003 +
            "\n"
        );
        return;
    }

    if (isMerging_3003 && !copying_3003) {
        if (i_3003 <= mid_3003 && j_3003 <= right_3003) {
            labelArray_3003[i_3003].setBackground(Color.CYAN);
            labelArray_3003[j_3003].setBackground(Color.CYAN);

            if (array_3003[i_3003] <= array_3003[j_3003]) {
                temp_3003[k_3003++] = array_3003[i_3003++];
            } else {
                temp_3003[k_3003++] = array_3003[j_3003++];
            }
        }

        stepArea_3003.append(
            "Langkah " +
            stepCount_3003++ +
            ": Bandingkan dan salin elemen\n"
        );
    }
}
```



```

        return;
    } else if (i_3003 <= mid_3003) {
        temp_3003[k_3003++] = array_3003[i_3003++];

        stepArea_3003.append(
            "Langkah " + stepCount_3003++ + ": Salin sisa kiri\n"
        );

        return;
    } else if (j_3003 <= right_3003) {
        temp_3003[k_3003++] = array_3003[j_3003++];

        stepArea_3003.append(
            "Langkah " + stepCount_3003++ + ": Salin sisa kanan\n"
        );

        return;
    } else {
        copying_3003 = true;
        k_3003 = 0;
        return;
    }
}

if (copying_3003 && k_3003 < temp_3003.length) {
    array_3003[left_3003 + k_3003] = temp_3003[k_3003];
    labelArray_3003[left_3003 + k_3003].setText(
        String.valueOf(temp_3003[k_3003])
    );
    labelArray_3003[left_3003 + k_3003].setBackground(Color.GREEN);
    k_3003++;

    stepArea_3003.append(
        "Langkah " + stepCount_3003++ + ": Tempelkan ke array utama\n"
    );
    return;
}

if (copying_3003 && k_3003 == temp_3003.length) {
    isMerging_3003 = false;
    copying_3003 = false;
}

if (mergeQueue_3003.isEmpty() && !isMerging_3003) {
    stepArea_3003.append("Selesai.\n");
    stepButton_3003.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
}
}

```

performStep_3003 yang berguna untuk memperlihatkan tahapan tahapan dari sorting hingga angka sudah terurut semuanya.

C. TUGAS

Instruksi

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (AZ) atau Durasi (terpendek ke terpanjang).

Pengerjaan

A. Lagu_2511533003.java

```
package pekan8_2511533003;

public class Lagu_2511533003 {

    String judul_3003;
    String penyanyi_3003;
    int durasi_3003;

    public Lagu_2511533003(
        String judul_3003,
        String penyanyi_3003,
        int durasi_3003
    ) {
        this.judul_3003 = judul_3003;
        this.penyanyi_3003 = penyanyi_3003;
        this.durasi_3003 = durasi_3003;
    }
}
```

Code diatas merupakan code ADT yang bernama Lagu_2511533003, code ini akan berfungsi sebagai ADT yang akan digunakan oleh driver nantinya. Memiliki 3 properti yang akan menjadi data serta penanda yang disimpan dari setiap ADT yang dibuat.

B. Sorting_2511533003.java

```
package pekan8_2511533003;
```

```

import java.util.Scanner;
public class Sorting_2511533003 {

    static void swap_3003(Lagu_2511533003[] arr_3003, int i_3003, int j_3003) {
        Lagu_2511533003 temp_3003 = arr_3003[i_3003];
        arr_3003[i_3003] = arr_3003[j_3003];
        arr_3003[j_3003] = temp_3003;
    }

    static void medianOfThree_3003(
        Lagu_2511533003[] arr_3003,
        int low_3003,
        int high_3003
    ) {
        int mid_3003 = low_3003 + (high_3003 - low_3003) / 2;

        if (arr_3003[low_3003].durasi_3003 > arr_3003[mid_3003].durasi_3003) {
            swap_3003(arr_3003, low_3003, mid_3003);
        }

        if (arr_3003[low_3003].durasi_3003 > arr_3003[high_3003].durasi_3003) {
            swap_3003(arr_3003, low_3003, high_3003);
        }

        if (arr_3003[mid_3003].durasi_3003 > arr_3003[high_3003].durasi_3003) {
            swap_3003(arr_3003, mid_3003, high_3003);
        }

        swap_3003(arr_3003, mid_3003, high_3003);
    }

    static int partition_3003(
        Lagu_2511533003[] arr_3003,
        int low_3003,
        int high_3003
    ) {
        medianOfThree_3003(arr_3003, low_3003, high_3003);
        int pivot_3003 = arr_3003[high_3003].durasi_3003;
        int i_3003 = (low_3003 - 1);

        for (int j_3003 = low_3003; j_3003 <= high_3003 - 1; j_3003++) {
            if (arr_3003[j_3003].durasi_3003 < pivot_3003) {
                i_3003++;
                swap_3003(arr_3003, i_3003, j_3003);
            }
        }
        swap_3003(arr_3003, i_3003 + 1, high_3003);
        return i_3003 + 1;
    }
}

```

```

public static void quickSort_3003(
    Lagu_2511533003[] arr_3003,
    int low_3003,
    int high_3003
) {
    if (low_3003 < high_3003) {
        int pi_3003 = partition_3003(arr_3003, low_3003, high_3003);
        quickSort_3003(arr_3003, low_3003, pi_3003 - 1);
        quickSort_3003(arr_3003, pi_3003 + 1, high_3003);
    }
}

static Lagu_2511533003[] mergeSort_3003(Lagu_2511533003[] arr_3003) {
    if (arr_3003.length == 1) {
        return arr_3003;
    }
    int half_3003 = arr_3003.length / 2;

    // splitting
    Lagu_2511533003[] left_split_3003 = new Lagu_2511533003[half_3003];
    Lagu_2511533003[] right_split_3003 =
        new Lagu_2511533003[arr_3003.length - half_3003];
    for (int i_3003 = 0; i_3003 < arr_3003.length; i_3003++) {
        if (i_3003 < half_3003) {
            left_split_3003[i_3003] = arr_3003[i_3003];
        } else {
            right_split_3003[i_3003 - half_3003] = arr_3003[i_3003];
        }
    }

    Lagu_2511533003[] left_arr_3003 = mergeSort_3003(left_split_3003);
    int left_len_3003 = left_arr_3003.length;
    int left_idx_3003 = 0;

    Lagu_2511533003[] right_arr_3003 = mergeSort_3003(right_split_3003);
    int right_len_3003 = right_arr_3003.length;
    int right_idx_3003 = 0;

    int idx_3003 = 0;
    Lagu_2511533003[] result_3003 = new Lagu_2511533003[left_len_3003 +
        right_len_3003];
    while (true) {
        if (left_idx_3003 >= left_len_3003) {
            while (right_idx_3003 < right_len_3003) {
                result_3003[idx_3003] = right_arr_3003[right_idx_3003];
                right_idx_3003 += 1;
                idx_3003 += 1;
            }
        }
    }
}

```

```

        break;
    } else if (right_idx_3003 >= right_len_3003) {
        while (left_idx_3003 < left_len_3003) {
            result_3003[idx_3003] = left_arr_3003[left_idx_3003];
            left_idx_3003 += 1;
            idx_3003 += 1;
        }
        break;
    }
}

Lagu_2511533003 left_value_3003 = left_arr_3003[left_idx_3003];
Lagu_2511533003 right_value_3003 = right_arr_3003[right_idx_3003];

int cmp = left_value_3003.judul_3003
    .substring(0, 1)
    .compareToIgnoreCase(
        right_value_3003.judul_3003.substring(0, 1)
    );

if (cmp <= 0) {
    result_3003[idx_3003] = left_value_3003;
    left_idx_3003 += 1;
} else {
    result_3003[idx_3003] = right_value_3003;
    right_idx_3003 += 1;
}
idx_3003 += 1;
}

return result_3003;
}

public static void shellSort(Lagu_2511533003[] A_3003) {
    int n_3003 = A_3003.length;
    int gap_3003 = n_3003 / 2;
    while (gap_3003 > 0) {
        for (int i_3003 = gap_3003; i_3003 < n_3003; i_3003++) {
            Lagu_2511533003 temp_3003 = A_3003[i_3003];
            int j_3003 = i_3003;

            while (
                j_3003 >= gap_3003 &&
                A_3003[j_3003 - gap_3003].judul_3003
                    .substring(0, 1)
                    .compareToIgnoreCase(
                        temp_3003.judul_3003.substring(0, 1)
                    ) > 0
            ) {
                A_3003[j_3003] = A_3003[j_3003 - gap_3003];

```

```

        j_3003 = j_3003 - gap_3003;
    }
    A_3003[j_3003] = temp_3003;
}

gap_3003 = gap_3003 / 2;
}
}

static int inputData_3003(Lagu_2511533003[] array_3003, int len_3003) {
    if (len_3003 == 20) {
        System.out.println("Jumlah Lagu Sudah Maksimal");
        return len_3003;
    }

    Scanner scanner_3003 = new Scanner(System.in);

    System.out.println("Masukkan Data Lagu");
    System.out.println("Masukkan Judul");
    String judul_3003 = scanner_3003.nextLine();

    System.out.println("Masukkan Judul");
    String penyanyi_3003 = scanner_3003.nextLine();

    System.out.println("Masukkan Durasi");
    int durasi_3003 = scanner_3003.nextInt();

    System.out.println("Data Berhasil Dibuat");

    array_3003[len_3003] = new Lagu_2511533003(
        judul_3003,
        penyanyi_3003,
        durasi_3003
    );

    len_3003 += 1;
    return len_3003;
}

static void tampilData_3003(Lagu_2511533003[] array_lagu_3003) {
    for (int i_3003 = 0; i_3003 < array_lagu_3003.length; i_3003++) {
        System.out.println(
            i_3003 +
            ". " +
            array_lagu_3003[i_3003].judul_3003 +
            " - " +
            array_lagu_3003[i_3003].durasi_3003 +
            " detik"
        );
    }
}

```

```

    }
}

static Boolean pilihAlgo_3003(Lagu_2511533003[] array_3003, int len_3003)
{
    if (len_3003 == 0) {
        System.out.print("Data Lagu Kosong");
        return false;
    }

    Scanner scanner_3003 = new Scanner(System.in);
    System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
    int algo_3003 = scanner_3003.nextInt();

    Lagu_2511533003[] array_lagu_3003 = new Lagu_2511533003[len_3003];
    for (int i_3003 = 0; i_3003 < len_3003; i_3003++) {
        array_lagu_3003[i_3003] = array_3003[i_3003];
    }
    System.out.println("\nData Sebelum Sorting: ");
    tampilData_3003(array_lagu_3003);

    if (algo_3003 == 1) {
        shellSort(array_lagu_3003);
    } else if (algo_3003 == 2) {
        quickSort_3003(array_lagu_3003, 0, array_lagu_3003.length - 1);
    } else if (algo_3003 == 3) {
        array_lagu_3003 = mergeSort_3003(array_lagu_3003);
    } else {
        System.out.println("Opsi tidak tersedia");
        return false;
    }

    System.out.println("\nData Setelah Sorting: ");
    tampilData_3003(array_lagu_3003);

    return true;
}

public static void main(String[] args) {
    Lagu_2511533003[] array_3003 = new Lagu_2511533003[20];
    int len_3003 = 0;
    Scanner scanner_3003 = new Scanner(System.in);

    while (true) {
        System.out.println("\n=== Sorting Playlist NIM: 2511533003===");
        System.out.println("1. Masukkan Data");
        System.out.println("2. Mulai Sorting");
        System.out.println("3. Exit");
        System.out.print("pilihan: ");
    }
}

```

```

int option_3003 = scanner_3003.nextInt();

if (option_3003 == 1) {
    len_3003 = inputData_3003(array_3003, len_3003);
} else if (option_3003 == 2) {
    if (pilihAlgo_3003(array_3003, len_3003)) {
        break;
    }
} else if (option_3003 == 3) {
    break;
} else {
}
}
}
}

```

Code diatas adalah kode yang menggunakan ADT Lagu yang telah dibuat, memiliki tujuan utama yaitu mengurutkan list lagu yang telah dimasukkan. Saat dijalankan akan muncul opsi berikut ini

```

=== Sorting Playlist NIM: 2511533003 ===
1. Masukkan Data
2. Mulai Sorting
3. Exit
pilihan: 

```

Terdapat 3 opsi, opsi pertama berfungsi untuk memasukkan data lagu baru yang mana program akan meminta imputan user untuk mengisi data lagu.

```

pilihan: 1
Masukkan Data Lagu
Masukkan Judul
a
Masukkan Judul
a
Masukkan Durasi
1
Data Berhasil Dibuat

=== Sorting Playlist NIM: 2411532014 ===
1. Masukkan Data
2. Mulai Sorting
3. Exit
pilihan: 

```


Gambar diatas adalah contoh memasukkan data lagu. Pada opsi kedua terdapat opsi untuk melakukan sorting, terdapat 3 jenis sorting antara lain:

1. Shell Sort, yang akan sorting huruf pertama alfabet A - Z
2. Quick Sort, yang akan sorting durasi
3. Merge Sort, yang akan sorting huruf pertama alfabet A - Z

Shell sort, digunakan untuk sorting alfabet pada judul lagu, sebagaimana pada gambar dibawah ini.

```
=== Sorting Playlist NIM: 2411532014 ===
1. Masukkan Data
2. Mulai Sorting
3. Exit
pilihan: 2
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 1

Data Sebelum Sorting:
0. a - 1 detik
1. f - 1 detik
2. o - 1 detik
3. p - 1 detik
4. d - 1 detik
5. b - 1 detik
6. g - 0 detik

Data Setelah Sorting:
0. a - 1 detik
1. b - 1 detik
2. d - 1 detik
3. f - 1 detik
4. g - 0 detik
5. o - 1 detik
6. p - 1 detik
```

Quick Sort berfungsi sorting berdasarkan durasi lagu, seperti yang tertera di bawah ini.

```

=== Sorting Playlist NIM: 2411532014 ===
1. Masukkan Data
2. Mulai Sorting
3. Exit
pilihan: 2
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:
0. a - 9 detik
1. b - 3 detik
2. c - 1 detik
3. d - 8 detik
4. e - 4 detik
5. f - 7 detik
6. 1 - 5 detik

Data Setelah Sorting:
0. c - 1 detik
1. b - 3 detik
2. e - 4 detik
3. 1 - 5 detik
4. f - 7 detik
5. d - 8 detik
6. a - 9 detik

```

Merge sort berfungsi sorting berdasarkan alfabet pertama pada judul lagu, seperti yang tertera pada gambar dibawah ini.

```

=== Sorting Playlist NIM: 2411532014 ===
1. Masukkan Data
2. Mulai Sorting
3. Exit
pilihan: 2
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 3

Data Sebelum Sorting:
0. a - 1 detik
1. v - 1 detik
2. c - 1 detik
3. o - 1 detik
4. e - 1 detik
5. z - 1 detik
6. h - 1 detik

Data Setelah Sorting:
0. a - 1 detik
1. c - 1 detik
2. e - 1 detik
3. h - 1 detik
4. o - 1 detik
5. v - 1 detik
6. z - 1 detik

```

BAB 3

PENUTUPAN

a. Kesimpulan

Algoritma merupakan bagian penting dalam sebuah arsitektur program terutama dalam program sorting, memilih sesuai dengan kebutuhan dan kasus akan membuat program akan lebih optimal dalam mengerjakan dan memecahkan masalah.

b. Saran

Laporan praktikum yang telah penulis tuliskan, masih memiliki kekurangan. Penulis sendiri pun juga membuka saran dan kritikan untuk meningkatkan kualitas laporan pratikum ini.

DAFTAR PUSTAKA

D3 Sistem Informasi Akuntansi, Telkom University. “Algoritma Pemograman: Pengertian, Fungsi Dan Jenis-Jenisnya.” Dac.Telkomuniversity.Ac.Id, diakses 3 Jun 2026, <https://dac.telkomuniversity.ac.id/algoritma-pemograman-pengertian-fungsi-dan-jenis-jenisnya/>.